C++ Notes: Getting User Input

(Summary from the Input section of the course)

Author: Ba Tung Le

Learning Objectives

After this section, you will be able to:

- Read integers, decimals, characters and full lines of text from the user.
- Prompt the user and store input in variables.
- Understand the difference between `cin` and `getline()`.
- Avoid the common `getline` / `cin` interaction pitfall using `cin.ignore()`.
- Get multiple values and do basic input validation.

Core Idea

Core idea

User input lets a program receive data at runtime. Always store input into variables so you can use that information later.

Basic Input with `cin`

Listing 1: Prompt for an integer and read it

```
int age;
cout << "Enter your age: ";
cin >> age;
cout << "You are " << age << " years old." << endl;
```

Example run (user types ‘30‘):
Output:

```
Enter your age: 30
You are 30 years old.
```

Note

`cin >>` reads a single token (number, char, or a single word). It skips leading whitespace and stops at the next whitespace.

Reading Different Types

Read a double

```
double d;
cout << "Enter a decimal number: ";
cin >> d;
cout << "You entered: " << d << endl;
```

Output (user types ‘4.5‘):

```
Enter a decimal number: 4.5
You entered: 4.5
```

Read a char

```
char grade;
cout << "Enter a grade (single letter): ";
cin >> grade;
cout << "Grade: " << grade << endl;
```

Output (user types ‘A‘):

```
Enter a grade (single letter): A
Grade: A
```

Reading a Full Line: `getline()`

```
string name;
cout << "Enter your full name: ";
getline(cin, name);
cout << "Hello, " << name << "!" << endl;
```

Output (user types ‘John Smith‘):

```
Enter your full name: John Smith
Hello, John Smith!
```

When to use `getline`

Use `getline(cin, var)` when you want the entire line (including spaces), e.g. full names, sentences.

Common Pitfall: `cin` then `getline`

If you use `cin >>` and then immediately call `getline`, the leftover newline from `cin` will make `getline` return an empty string.

Listing 2: Problem example

```
int age;
string name;

cin >> age; // user types: 30
getline(cin, name); // reads the leftover newline => name == ""
```

Output (unexpected):

```
Enter age: 30
Enter name:
Hello, !
```

Fix with `cin.ignore()`

After `cin >>`, call `cin.ignore()` to discard the leftover newline before `getline`:

```
cin >> age;
cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
getline(cin, name);
```

Multiple Inputs on One Line

```
int a, b;
cout << "Enter two integers separated by space: ";
cin >> a >> b;
cout << "a=" << a << ", b=" << b << endl;
```

User input: ‘4 7‘ **Output:**

```
Enter two integers separated by space: 4 7
a=4, b=7
```

Basic Validation Example

Listing 3: Simple integer validation

```
int age;
cout << "Enter your age: ";
while (!(cin >> age) || age < 0) {
  cout << "Invalid input. Enter a non-negative integer: ";
  cin.clear(); // clear error state
  cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // discard bad
      input
}
cout << "You entered: " << age << endl;
```

Notes:

- `cin.clear()` resets the stream state after a failed extraction.
- `cin.ignore(...)` discards the rest of the bad input.

Tips and Good Practices

Tips

- Always prompt the user — clear prompts reduce mistakes.
- Validate numeric input before using it.
- Use `getline` for multi-word input (names, sentences).
- When mixing `cin >>` and `getline`, use `cin.ignore()` as shown.
- For robust input loops, always clear and ignore on failure.

Common Mistakes

Common mistakes

- Forgetting to prompt the user.
- Expecting `getline` to work immediately after `cin >>` without `cin.ignore()`.
- Not validating user input (leading to crashes or wrong results).
- Assuming the user will enter the right type (always check).

Ba Tung Le
Email: tombn2607@gmail.com